🌊 Advanced WaveWidget (Flow / Ribbon Patterns)

A highly customizable **animated wave background** built with `CustomPainter`, supporting **multiple motion patterns** such as organic flow-fields and ribbon drift. Designed to replicate modern abstract wave backgrounds like your reference image.

✨ Key Highlights

- Multiple **motion patterns** (classic, flow-field, ribbon drift)
- Organic, non-repeating movement (no mechanical sine look)
- Layered ultra-thin wave lines for depth
- Vertical spread (not locked to a single center line)
- Dual opposite-flow wave layers
- Fully configurable and extensible

🍂 File Structure

```
lib/
 ├─ wave_widget.dart
 ├─ wave_painter.dart
 └─ wave_motion_pattern.dart
```

🪱 Motion Patterns

```dart
enum WaveMotionPattern {
  classic,     // simple sine wave
  flowField,   // organic, reference-like motion
  ribbonDrift, // soft horizontal drifting ribbons
}
```

Pattern Comparison

Pattern	Visual Style	Best Use Case
`classic`	Regular sine waves	Simple / low-cost animation
`flowField`	Organic, fluid, modern	Best match to abstract UI backgrounds
`ribbonDrift`	Smooth drifting ribbons	Headers, hero sections

WaveWidget Parameters

Property	Type	Default	Description
height	double	200	Height of the wave area
gradientColors1	List<Color>	required	Gradient for wave layer 1
gradientColors2	List<Color>	required	Gradient for wave layer 2
lineCount	int	60	Number of lines per layer
amplitude	double	28	Vertical wave strength
waveLength	double	220	Horizontal wavelength
speed	double	0.6	Animation speed
pattern	WaveMotionPattern	flowField	Motion behavior

How the Motion Works

Each wave line is computed using a **modified sine function**:

```
y = centerY
  + verticalOffset
  + drift
  + localAmplitude × sin(2π(x / waveLength) + phase)
```

What Makes It Organic

- **Vertical offset** per line → layered ribbons
- **Local amplitude scaling** → depth illusion
- **Time-based drift** → removes rigid looping
- **Opposite-direction layers** → richer flow

🌳Recommended Presets

🌀Best Match (Reference Image)

```
WaveWidget(
  height: 220,
  pattern: WaveMotionPattern.flowField,
```

```
  lineCount: 60,
  amplitude: 28,
  speed: 0.6,
)
```

🎗️Soft Ribbon Header

```
pattern: WaveMotionPattern.ribbonDrift,
speed: 0.5,
lineCount: 45
```

🌊Minimal / Performance Mode

```
pattern: WaveMotionPattern.classic,
lineCount: 25,
amplitude: 16
```

🍁Performance Notes

- Reduce `lineCount` on low-end devices
- Keep stroke widths under `1.0`
- Prefer slower speeds for elegance
- Wrap widget in `RepaintBoundary` when used in complex UIs

🦏Full Source Code

wave_motion_pattern.dart

```
enum WaveMotionPattern {
  classic,
  flowField,
  ribbonDrift,
}
```

wave_widget.dart

```
import 'package:flutter/material.dart';
import 'wave_painter.dart';
import 'wave_motion_pattern.dart';
```

```dart
class WaveWidget extends StatefulWidget {
  final double height;
  final List<Color> gradientColors1;
  final List<Color> gradientColors2;
  final int lineCount;
  final double amplitude;
  final double waveLength;
  final double speed;
  final WaveMotionPattern pattern;

  const WaveWidget({
    super.key,
    this.height = 200,
    required this.gradientColors1,
    required this.gradientColors2,
    this.lineCount = 60,
    this.amplitude = 28,
    this.waveLength = 220,
    this.speed = 0.6,
    this.pattern = WaveMotionPattern.flowField,
  });

  @override
  State<WaveWidget> createState() => _WaveWidgetState();
}

class _WaveWidgetState extends State<WaveWidget>
    with SingleTickerProviderStateMixin {
  late final AnimationController _controller;

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(
      vsync: this,
      duration: Duration(milliseconds: (4000 ~/ widget.speed)),
    )..repeat();
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
```

```
    return SizedBox(
      height: widget.height,
      width: double.infinity,
      child: AnimatedBuilder(
        animation: _controller,
        builder: (_, __) {
          return CustomPaint(
            painter: WavePainter(
              animationValue: _controller.value,
              gradientColors1: widget.gradientColors1,
              gradientColors2: widget.gradientColors2,
              lineCount: widget.lineCount,
              amplitude: widget.amplitude,
              waveLength: widget.waveLength,
              pattern: widget.pattern,
            ),
          );
        },
      ),
    );
  }
}
```

wave_painter.dart

```
import 'dart:math';
import 'package:flutter/material.dart';
import 'wave_motion_pattern.dart';

class WavePainter extends CustomPainter {
  final double animationValue;
  final List<Color> gradientColors1;
  final List<Color> gradientColors2;
  final int lineCount;
  final double amplitude;
  final double waveLength;
  final WaveMotionPattern pattern;

  WavePainter({
    required this.animationValue,
    required this.gradientColors1,
    required this.gradientColors2,
    required this.pattern,
    this.lineCount = 60,
    this.amplitude = 28,
    this.waveLength = 220,
```

```
});

@override
void paint(Canvas canvas, Size size) {
  final paint = Paint()
    ..style = PaintingStyle.stroke
    ..strokeCap = StrokeCap.round;

  final centerY = size.height / 2;
  final time = animationValue * 2 * pi;

  for (int layer = 0; layer < 2; layer++) {
    for (int i = 0; i < lineCount; i++) {
      final path = Path();
      final p = i / lineCount;

      final verticalOffset = (p - 0.5) * size.height * 0.7;
      final localAmplitude = amplitude * (0.4 + p);
      final basePhase = p * 2 * pi;

      for (double x = 0; x <= size.width; x++) {
        double phase;
        double drift;

        switch (pattern) {
          case WaveMotionPattern.classic:
            phase = time + basePhase;
            drift = 0;
            break;

          case WaveMotionPattern.flowField:
            phase = (layer == 0 ? 1 : -1) * time * (0.6 + p) +
                sin(time * 0.3 + p * 4);
            drift = sin(time * 0.4 + x * 0.01) * 8;
            break;

          case WaveMotionPattern.ribbonDrift:
            phase = time * 0.5 + basePhase;
            drift = cos(time + p * 6) * 12;
            break;
        }

        final y = centerY +
            verticalOffset +
            drift +
            localAmplitude *
                sin((x / waveLength) * 2 * pi + phase);
```

```
          x == 0 ? path.moveTo(x, y) : path.lineTo(x, y);
        }

        paint
          ..strokeWidth = layer == 0 ? 0.6 : 0.5
          ..shader = LinearGradient(
            begin: layer == 0
                ? Alignment.topLeft
                : Alignment.bottomLeft,
            end: layer == 0
                ? Alignment.bottomRight
                : Alignment.topRight,
            colors: (layer == 0 ? gradientColors1 : gradientColors2)
                .map((c) => c.withOpacity(0.1 + p * 0.35))
                .toList(),
          ).createShader(Offset.zero & size);

        canvas.drawPath(path, paint);
      }
    }
  }

  @override
  bool shouldRepaint(covariant CustomPainter oldDelegate) => true;
}
```

🦜 Summary

This implementation allows you to **swap motion behavior without rewriting rendering logic**, making it ideal for design experimentation and reusable UI libraries.

If you want: - Runtime pattern switching - Perlin-noise motion - Filled ribbon bands - Scroll-reactive waves

Just tell me.